

# JJBike

## 1. Objective.

Build a data warehouse so that the administrator of the fictional company JJBike can understand their business.

## 2. Modeling.

To create the analytical database (OLAP) structure model based on the transactional database (OLTP), using the Star Schema model (the most common model for Business Intelligence), the following transformations were made:

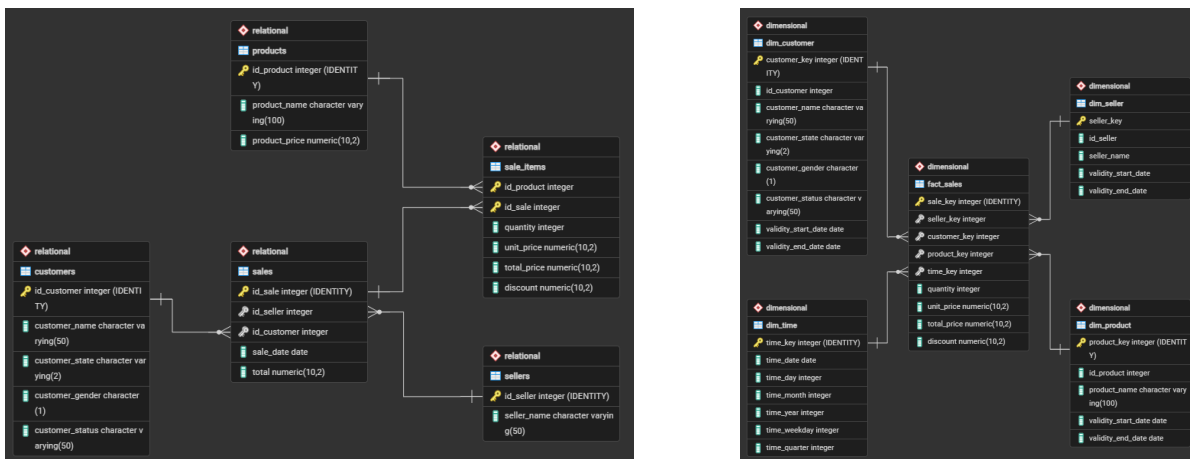
1. Since the goal is to describe the business model, all tables in the OLTP environment, that is, the tables that have one-to-many (1:N) relationships with the table *sales*, were transformed into dimensions:

- **Product ('what?'):** *dim\_product*, which holds the attributes of the *products* table from the OLTP environment (excluding *product\_price*, as that stays in the fact table), plus the addition of *validity\_start\_date* and *validity\_end\_date* for versioning;
- **Seller ('who?'):** *dim\_seller*, which holds the attributes of the *sellers* table from the OLTP environment, plus the addition of *validity\_start\_date* and *validity\_end\_date* for versioning;
- **Customer ('who?'):** *dim\_customer*, which holds the attributes of the *customers* table from the OLTP environment, plus the addition of *validity\_start\_date* and *validity\_end\_date* for versioning.

2. **The time dimension ('when?') was created:** *dim\_time*, which holds the date attribute from the *sales* table in the transactional environment, plus the derived attributes *time\_day*, *time\_month*, *time\_year*, *time\_weekday*, and *time\_quarter*, created from *time\_date*, to enable dimension hierarchies in analyses;

3. In the OLAP environment, the fact table *fact\_sales* consolidated the header table (*sales*) and the detail table (*sale\_items* (N:M between sales and products)), from the OLTP environment, into a single granularity level: the product item per sale. With its Surrogate Key (SK) as Primary Key (PK), the dimension SKs as Foreign Keys (FK), and the attributes of the junction table *sale\_items* as metrics.

- The granularity of the fact table *fact\_sales* was defined at the line item per sale level, ensuring the highest level of detail possible for the JJBike administrator's analyses.



### 3. Transactional Environment (OLTP) - Microsoft SQL Server Management Studio (SSMS).

In Microsoft SQL Server Management Studio, the 'JJBike' database was created.

Note: Throughout the SQL code, the Snake Case naming convention will be used.

1. The *Relational* database was created, serving as the data source;
2. The operational tables *sellers*, *products*, *customers*, *sales*, and *sale\_items* were created in the *Relational* database.
  - Just as in the analytical environment, the primary keys of the relational tables were configured with the *IDENTITY* property, ensuring native auto-increment by the DBMS;
  - All columns, except the Primary Keys and *validity\_end\_date*, were defined with *NOT NULL* constraints, enforcing data integrity at the database level;
  - Foreign keys (FKs) were implemented to ensure data consistency between tables:
    - The *sales* and *sale\_items* tables have strict relationships with the registration tables (*customers*, *sellers*, and *products*), preventing the creation of orphan records in the system;
    - The default constraint when defining a FK without specifying a delete or update action is *NO ACTION*, which, if an attempt is made to delete a record in the parent table that has dependents in the child table, blocks the operation and raises an error, preventing the creation of orphan data.
3. Inserts were performed into the tables *sellers*, *products*, *customers*, *sales*, and *sale\_items*.
  - **Note:** The *SET DATEFORMAT dmy* command tells the database that the date format being sent is Day/Month/Year, so for example for 25/04/2026, the DBMS correctly understands that 25 is the day and 04 is the month of April, executing the *INSERT* successfully.

#### Files used (1. Environments/1. OLTP):

- **Points 1 and 2:** 1. *Scripts\_Create\_Table\_Relational.sql*;
- **Point 3:** 2. *Insert\_Into\_Customers.sql*, 3. *Insert\_Into\_Products.sql*, 4. *Insert\_Into\_Sellers.sql*, 5. *Insert\_Into\_Sales.sql* and 6. *Insert\_Into\_SaleItems.sql*.

### 4. Analytical Environment (OLAP) - Microsoft SQL Server Management Studio (SSMS).

In Microsoft SQL Server Management Studio, the 'JJBike' database was used.

Note: Throughout the SQL code, the snake\_case naming convention will be used.

#### 4.1. Analytical Database (OLAP)

1. The *Dimensional* database was created, representing the analytical environment (Data Warehouse (DW)) structured in a Star Schema model;
2. The dimensions *dim\_seller*, *dim\_customer*, *dim\_product*, *dim\_time*, and the fact table *fact\_sales* were created in the *Dimensional* database.

- The *IDENTITY* property in SQL Server is used to automatically generate the SKs as the internal primary keys of the DW in an incrementally growing sequence;
- All columns, except the Primary Keys and *validity\_end\_date*, were defined with *NOT NULL* constraints, enforcing data integrity at the database level;
- The concept of Slowly Changing Dimensions (SCD) Type 2 was applied to the sellers, customers, and products dimensions through the columns *validity\_start\_date* and *validity\_end\_date*, enabling full versioning and the preservation of data history over time.

3. The time dimension was loaded. This process was carried out through three main structures:

- The *DECLARE* block defines where the count starts, where it ends, and the increment step:
  - **@start\_date and @end\_date:** The script maps the period from January 1st 1970 to December 31st 2030. (The YYYYMMDD format is used for safety, as SQL Server reads it correctly regardless of the server language);
  - **@step = 1:** Indicates that the advance will be made 1 day at a time.
- The *WITH date\_sequence AS* block uses a resource called Recursive CTE (Common Table Expression) to create the following loop:
  - **WITH date\_sequence (Recursive CTE):** Works as an automatic calendar generator, feeding the code with a virtual table containing all possible dates between the defined start and end dates;
  - **SELECT:** Slices each of these generated dates to extract and map useful information for analysis, such as the day, month, year, weekday, and quarter;
  - **OPTION (MAXRECURSION 0):** Ensures that the database ignores the default recursion limit, allowing the internal loop to process and insert all the thousands of dates at once in perfect chronological order.
- The *ORDER BY 1* block ensures that the time dimension table is inserted in perfect chronological order, from the oldest to the most recent date.

4. The following data loads from the transactional database (*Relational*) into the analytical database (*Dimensional*) were performed in the described order:

- **Initial Load of customers, products, and sellers:**
  - **Date helpers:** The code begins by declaring two global time variables (*@dt\_end* and *@dt\_start*) with a 3-millisecond difference between them. They ensure that all rows processed in the same batch share exactly the same timestamps, avoiding discrepancies in the DW;
  - **MERGE logic (SCD Type 2):** Instead of using separate CTEs to update and insert, the process uses the native *MERGE* statement, comparing the source table (*USING*) with the destination dimension (*AS T*). The join is made by matching the source ID and the *T.validity\_end\_date IS NULL* filter, ensuring the comparison occurs only against the currently active history;
  - **Handling changes and new records:** If the ID matches but some attribute (such as *seller\_name*, *customer\_state*, *customer\_gender*, or *customer\_status*) has changed, the *MERGE* executes a *THEN UPDATE* setting the end date to *@dt\_end* (closing the old history). If the ID is not found in the destination (*WHEN NOT MATCHED BY TARGET*), the *MERGE* automatically executes a *THEN INSERT* creating the record with start date equal to *@dt\_start* and end date as *NULL*;
  - **OUTPUT (new history generation):** To complete the SCD Type 2 logic (which requires two actions for changed records), the *OUTPUT* clause captures the

*\$ACTION* of everything that happened in the inner block. An outer *INSERT INTO ... SELECT* wraps the *MERGE* and, using the filter *WHERE O.action = 'UPDATE'*, captures only the rows that were closed and inserts a new active version with updated data and start date at *@dt\_start*.

- **Fact sales load (January, February, March, and remaining months):**
  - **Insertion logic:** The code uses an *INSERT INTO ... SELECT* that consolidates transactional data directly into the fact table;
  - **Key mapping (SKs):** Performs *INNER JOIN* between the *sales* and *sale\_items* relational tables and the DW dimensions to capture the SKs and metric attributes. The crucial rule applied in each dimensional join is the *validity\_end\_date IS NULL* filter. This ensures that the sale is associated with the SK version of the customer, product, or seller that was active and valid at the exact moment the load was executed;
  - **Time filter:** Uses the function *MONTH(v.sale\_date)* in the *WHERE* clause to segment and isolate the fact load month by month (January = 1, February = 2, March = 3, and remaining months with > 3).
- **Status Update (SCD Type 2 Simulation):**
  - **Relational change:** A simple *UPDATE* is made directly on the source table (*Relational..customers*) simulating a real-world data change;
  - **History processing:** When the same customer load block is run again, the date variables are redeclared due to the batch scope (*GO*). The *MERGE* detects that *customer\_status* from the source differs from the *customer\_status* stored in *dim\_customer*. This triggers the *MATCHED* condition, closing the old record with the end date and passing the changed record to *OUTPUT*, which creates the new active row preserving the perfect history of what the customer was before becoming 'Gold' or 'Platinum'.
- **Results Verification:**
  - **Audit Queries:** The script ends with *SELECT* queries combining *fact\_sales* with *dim\_customer* and *dim\_time*. This physically proves the SCD Type 2 behavior: January sales remain associated with the old SKs (snapshot of that time), while February onwards already point to the new SK with the updated status.

#### ***Files used (1. Environments/2. OLAP):***

- **Points 1 and 2:** 1. *Scripts\_Create\_Table\_Dimensional.sql*;
- **Point 3:** 2. *Insert\_Into\_dim\_time.sql*;
- **Point 4:** 3. *Script.sql*.

#### ***4.2. KPI.***

Microsoft SQL Server Management Studio (SSMS) is not a dimensional data server, therefore a denormalized table (a single table with all data) was created to measure total revenue for the fictional company JJBike. This process was carried out through four main structures:

- The *SELECT* block selects only the columns of interest for the macro analysis, extracting the time attribute (month), calculating the aggregated total revenue metric (*SUM*) from the fact table, and dynamically generating the target column through a conditional structure (*CASE*);

- The conversion function *CAST(... AS NUMERIC(18,2))*, which wraps the entire *CASE* block, replaces the *::numeric* operator from PostgreSQL. It is used to force and standardize the data type of the target column as an exact decimal value with two decimal places, ensuring perfect mathematical compatibility with the revenue metric when calculating achievement targets;
- The *INTO* block defines the physical creation of the new table in the database, specifying *Dimensional..kpi* as the final destination that will store the consolidated results and targets data per month;
- The *FROM* block connects the central *Dimensional..fact\_sales* table to the *Dimensional..dim\_time* dimension through an *INNER JOIN* using the surrogate keys (SKs), grouping (*GROUP BY*) and ordering (*ORDER BY*) the final results by month.

#### **Files used (1. Environments/2. OLAP/):**

- 4. *KPI.sql*.

### **5. Reports - Power BI.**

1. The tables of the *Dimensional* database, created in Microsoft SQL Server Management Studio, were connected to Power BI.

2. The following changes were made to the tables:

- In *fact\_sales*, a column called *discount\_percent* (with two decimal places) was created, with the following formula: *discount\_percent = TRUNC(('dimensional fact\_sales'[discount] / 'dimensional fact\_sales'[total\_price]) \* 100, 2);*
- In *dim\_customer*, a column called *full\_location* was created, with the following formula:
  - *location\_map = SWITCH('dimensional dim\_customer'[customer\_state], "AC", "Acre", "AL", "Alagoas", "AM", "Amazonas", "AP", "Amapá", "BA", "Bahia", "CE", "Ceará", "DF", "Distrito Federal", "ES", "Espírito Santo", "GO", "Goiás", "MA", "Maranhão", "MG", "Minas Gerais", "MS", "Mato Grosso do Sul", "MT", "Mato Grosso", "PA", "Pará", "PB", "Paraíba", "PE", "Pernambuco", "PI", "Piauí", "PR", "Paraná", "RJ", "Rio de Janeiro", "RN", "Rio Grande do Norte", "RO", "Rondônia", "RR", "Roraima", "RS", "Rio Grande do Sul", "SC", "Santa Catarina", "SE", "Sergipe", "SP", "São Paulo", "TO", "Tocantins", "Unknown") & ", Brazil".*
- In *dim\_kpi*, three metrics were created so that, in the KPI report, the card behaves dynamically according to what is selected in the button slicer:
  - *measure\_target = IF(ISFILTERED('dimensional kpi'[month]) || ISFILTERED('dimensional dim\_time'[time\_year]), SUM('dimensional kpi'[target]), 0).*

```

○ measure_total_revenue =
  IF(
    ISFILTERED('dimensional kpi'[month]) || ISFILTERED('dimensional
    dim_time'[time_year]),
    SUM('dimensional kpi'[total_revenue]),
    0
  ).

```

3. The following reports were created:

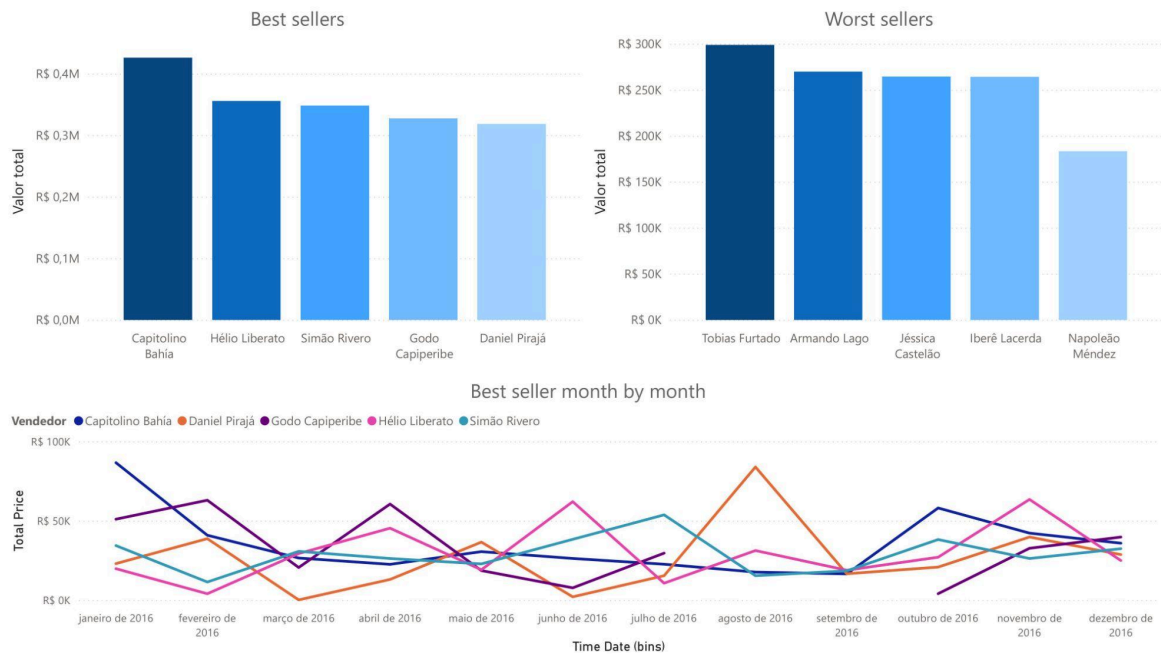
### **(Customers)**

The customers report shows: total value sum by state, total value sum by status, total value sum by gender, and total value sum by customer (top 5).



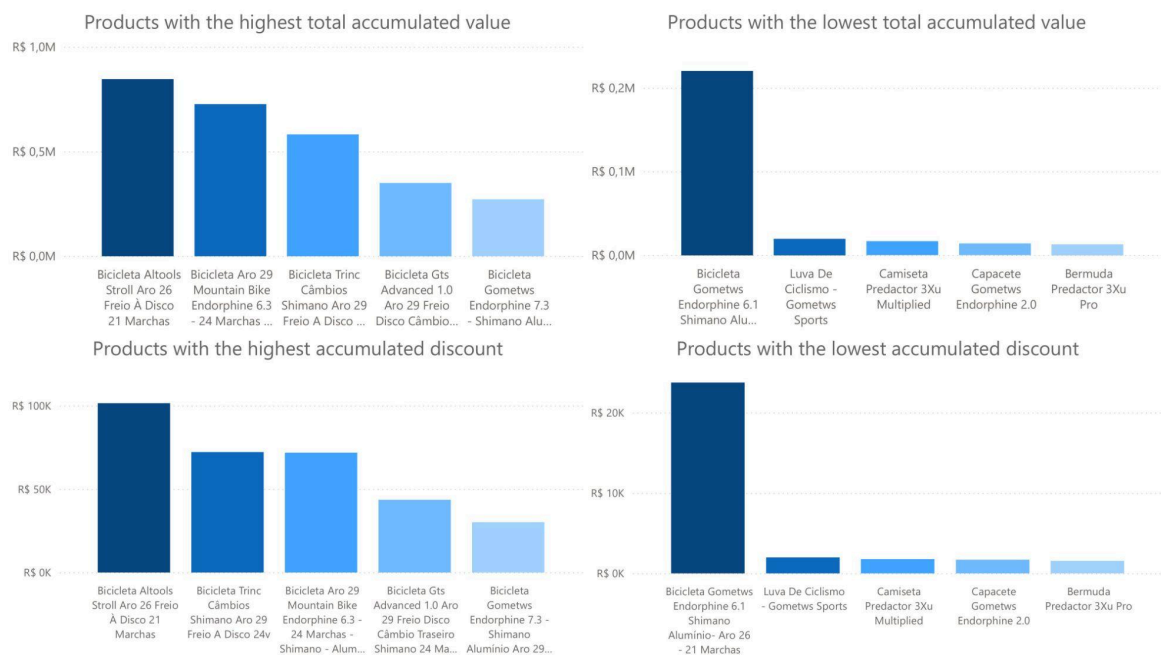
### **(Sellers)**

The sellers report shows: total value sum by seller (top 5 best sellers, top 5 worst sellers, and top 5 best sellers compared month by month).



### (Products)

The products report shows: total value sum by product (top 5 best-selling products and top 5 least-selling products) and discount sum by product (top 5 products with the most accumulated discount and top 5 with the least accumulated discount).



### (Sales)

The sales report shows: total value sum of products sold (compared month by month), quantity sum of products sold (compared month by month), discount sum of products sold (compared month by month), and product, seller, and customer cards for filtering.



### (KPI)

The KPI report shows: target, total acquired value, sum of target and total acquired value (compared month by month), and month and year cards for filtering.



*Generated files available in '2. Artifacts/1. Reports'.*

### 6. Cube - Microsoft Visual Studio.

1. The tables of the *Dimensional* database were structured and hosted on a full SQL Server instance (Developer Edition), ensuring native support for Business Intelligence (BI) features;

2. A Data Source View (DSV) was created, a visual environment where the fact and dimension tables were mapped (excluding the *kpi* table), generating a logical model in Star Schema format;

3. The following dimensions were created:

- ***dim\_customer***: *customer\_key*, *customer\_name*, *customer\_gender*, *customer\_state*, and *customer\_status*;
- ***dim\_product***: *product\_key* and *product\_name*;
- ***dim\_time***: *time\_key*, *time\_year*, *time\_quarter*, *time\_month*, and *time\_day*;
- ***dim\_seller***: *seller\_key* and *seller\_name*.

4. The following hierarchies were created from the dimensions:

- ***Geographic Hierarchy***: *customer\_state* → *customer\_name* (*dim\_customer*);
- ***Time Hierarchy***: *time\_year* → *time\_quarter* → *time\_month* → *time\_day* (*dim\_time*);
- Notes:
  - *customer\_gender* and *customer\_status* (from *dim\_customer*) should remain free in the side panel as Loose Dimension Attributes, as placing them inside *Geographic Hierarchy* would imply they are part of a geographic subdivision;
  - *dim\_seller* and *dim\_product* have only one attribute beyond their keys, therefore they did not generate hierarchies, and their attributes became Loose Dimension Attributes;
  - To avoid duplicate key errors in the Analysis Services engine during *dim\_time* processing, the KeyColumns properties were customized as composite keys, linking *time\_quarter* with *time\_year*, *time\_month* with *time\_year*, and *time\_day* with *time\_month* and *time\_year*, ensuring the uniqueness and integrity of the attribute relationship tree.

5. The cube was structured from the *fact\_sales* fact table. During the process, relationship keys and analytically incorrect fields (such as *unit\_price*, whose sum aggregation makes no business sense) were ignored. Only the real performance metrics were kept (*quantity*, *total\_price*, *discount*, and the automatic record count);

6. A successful deploy of the project was performed to the local Analysis Services server instance, compiling the structures and loading the data into memory;

7. Visual representation of the created cube:

| Customer State | Customer Name     | Time Year | Time Quarter | Time Month | Time Day | Quantity | Discount | Total Price |
|----------------|-------------------|-----------|--------------|------------|----------|----------|----------|-------------|
| AC             | Alarico Quinterno | 2016      | 4            | 12         | 10       | 1        | 60,45    | 155         |
| AC             | Antônio Sobral    | 2016      | 3            | 9          | 17       | 1        | 1593,36  | 8852        |
| AC             | Antônio Sobral    | 2016      | 4            | 10         | 16       | 2        | 1741,25  | 7935,6      |
| AC             | Basilio Soares    | 2016      | 1            | 3          | 29       | 1        | 4,65     | 155         |
| AC             | Basilio Soares    | 2016      | 2            | 4          | 17       | 9        | 879,08   | 22167,65    |
| AC             | Basilio Soares    | 2016      | 2            | 4          | 18       | 3        | 125,1    | 3289        |
| AC             | Basilio Soares    | 2016      | 2            | 5          | 14       | 1        | 64,07    | 2135,52     |
| AC             | Basilio Soares    | 2016      | 2            | 6          | 6        | 1        | 650,93   | 6509,3      |
| AC             | Basilio Soares    | 2016      | 2            | 6          | 24       | 1        | 16,92    | 188         |
| AC             | Basilio Soares    | 2016      | 3            | 7          | 5        | 2        | 775,25   | 7793        |
| AC             | Carla Briones     | 2016      | 4            | 12         | 5        | 1        | 587,75   | 2260,58     |
| AC             | Ester Castanho    | 2016      | 3            | 9          | 26       | 5        | 2327,33  | 15416,98    |
| AC             | Evaristo Bahia    | 2016      | 4            | 10         | 10       | 3        | 692,17   | 4836,7      |
| AC             | Irene Villanueva  | 2016      | 4            | 11         | 12       | 3        | 1654,34  | 7183,75     |
| AL             | Cid Pardo         | 2016      | 3            | 8          | 21       | 3        | 1769,02  | 13295       |
| AL             | Cid Pardo         | 2016      | 4            | 10         | 18       | 3        | 2430,31  | 13522,61    |
| AM             | Deise Farias      | 2016      | 3            | 8          | 22       | 3        | 1231,04  | 8976,8      |
| AM             | Deise Farias      | 2016      | 3            | 9          | 7        | 1        | 1472,16  | 9201        |
| AM             | Diana Baptista    | 2016      | 1            | 2          | 12       | 4        | 79,67    | 11112,58    |
| AM             | Diana Baptista    | 2016      | 1            | 2          | 12       | 1        | 1,60     | 160,2       |

**Generated files available in '2. Artifacts/2. Cube/'.**